COLLEGE BUSINESS EDUCATION
Dar es Salaam

INDIVIDUAL ASSIGNMENT

COURSE: BACHELOR DEGREE IN INFORMATION TECHINOLOGY.
MODULE: DATA STRUCTURES AND ALGORITHM
MODULE CODE: ITU07313
FACILITATOR: Dr. AHMED KIJAZI
YEAR OF STUDY: Second Year
SEMESTER : 1st
ACADEMIC YEAR: 2025/2026
STUDENT NAME: France Gaufrid Fussy
REGISTRATION NUMBER: 03.3154.01.01.2024
STREAM: TNC

Deadline Date: 2026-01-12

Question: Write a Java Program to implement a circular, singly linked list

A **circular singly linked list** is a variation of the singly linked list where the last node points back to the first node instead of `null`. This creates a circular structure that allows continuous traversal without hitting the end.

```java
import java.util.Arrays;

// Node class representing each fruit
class Fruit {
    String name;
    Fruit next;

    Fruit(String name) {
        this.name = name;
        this.next = null;
    }
}

class FruitList {
    private Fruit start = null;
    private Fruit end = null;

    public void add(String name) {
        Fruit newFruit = new Fruit(name);
        if (start == null) {
            start = newFruit;
            end = newFruit;
            end.next = start; // circular link start here to be implimented
        } else {
            end.next = newFruit;
            end = newFruit;
            end.next = start; // maintaining circular link
        }
    }

    // Display all fruits
    public void show() {
        if (start == null) {
            System.out.println("List is empty");
            return;
        }

        Fruit temp = start;
        do {
            System.out.print(temp.name + " ");
            temp = temp.next;
        } while (temp != start);
        System.out.println();
    }

```

```java
    // Deleting fruits by name
    public void remove(String name) {
        if (start == null) return;

        Fruit current = start;
        Fruit previous = end;

        do {
            if (current.name.equals(name)) {
                if (current == start) {
                    if (start == end) {
                        start = null;
                        end = null;
                    } else {
                        start = start.next;
                        end.next = start;
                    }
                } else if (current == end) {
                    end = previous;
                    end.next = start;
                } else {
                    previous.next = current.next;
                }
                return;
            }
            previous = current;
            current = current.next;
        } while (current != start);
    }


    public int size() {
        if (start == null) return 0;
        int count = 0;
        Fruit temp = start;
        do {
            count++;
            temp = temp.next;
        } while (temp != start);
        return count;
    }
}
```

```java

// Main class adding and Displaying Fruit l;ist
public class Main_1 {
    public static void main(String[] args) {
        FruitList list = new FruitList();

        Arrays.asList("Mango", "Banana", "Apple", "WaterMelon", "Passion")
              .forEach(list::add);

        System.out.println("Fruits list:");
        list.show();

        list.remove("Banana");
        System.out.println("After deleting Banana:");
        list.show();

        list.remove("WaterMelon");
        System.out.println("After deleting WaterMelon:");
        list.show();

        list.remove("Passion");
        System.out.println("After deleting Passion:");
        list.show();

        System.out.println("Total fruits in list is: " + list.size());
    }
}

```

Output:

```
┌──(fussy㉿kali)-[~/Documents]
└─$  cd /home/fussy/Documents ; /usr/bin/env /u
X:+ShowCodeDetailsInExceptionMessages -cp /home
3c5e519175b1bfe9422be57d400de7/redhat.java/jdt_
Fruits list:
Mango Banana Apple WaterMelon Passion
After deleting Banana:
Mango Apple WaterMelon Passion
After deleting WaterMelon:
Mango Apple Passion
After deleting Passion:
Mango Apple
Total fruits in list is: 2
```